

Monte Carlo Simulation of a Simple Polymer Chain

Implementation Report

Project_2026_II

12 April 2026

Contents

0.1 Project Overview

In this assignment, our goal was to develop a Monte Carlo simulation program to explore the conformational landscape of a 500-Carbon linear polymer chain (polyethylene) with varying torsional angles. The simulation generates the initial conformation (with & without Hydrogens), measures torsional energy rotations, end-to-end distance, and intramolecular Lennard-Jones interactions, and finally analyzes & visualizes the results.

After creating a serial processing version of the program, parallelization techniques involving both MPI and OpenMP were incorporated. These were compared to the serial version to demonstrate both their individual and combined High Performance Computing capabilities & benefits.

Many techniques taught in class were crucial in the development of this program, including: - **Git/Github** as a code repository & teamwork coordination platform - **Makefile** to automate the compilation and execution of the code - **Shell scripting** for facilitating server deployment - **MPI & openMP** to parallelize the code - **Python** for post-processing and plotting

0.2 Repository Structure

Github was used as our team's code repository management system, and from the root of the project, the code is divided among 3 main folders: **src**, **results**, and **docs**.

- **src** contains:
 - Makefile & shell scripts
 - Various versions of the main program (serial & parallel)
 - A **confs** folder where initial configurations are stored and compile-time options can be controlled via a file called **input.dat**
 - A **lib** folder where module source files, python analysis files, and plotting style dependencies reside (*further detail below*)
- **results** contains the output data from the simulations
- **docs** contains documentation and reports, including **img** folder for embedded images

In addition to these folders, a customary **README** file is included in the root directory detailing run instructions and code dependencies/requirements. The bulk of the code is written in Fortran, while the post-processing is done in Python.

Module Library Introduction The following modules support the *main...f90* files: - **energy.f90** & **energy_all_atoms.f90** - Permit **dual energy modes**, computing energetic contributions using either TraPPE-UA or OPLS-AA (with effective backbone potentials) depending on the **explicit_h** toggle dynamically read from **input.dat**. - **initial-conf.f90** - Generates initial configuration of the polymer chain. - **io.f90** - File input/output module. - **monte_carlo.f90** - Provides the rotation algorithm and subroutine MCS step used to update spatial configurations. - **observables.f90** - Computes the structural observables: End-to-end distance (R_{ee}), Radius of Gyration (R_g), and torsion angles. - **parameters.f90** - Stores global parameters abstracted from *main...f90* code. Works in tandem with **input.dat** settings. - **plot...py** files - Various scripts used for plotting observables and serial vs parallel code comparisons. - **requirements.txt** - List of python module dependencies. - **science.mplstyle** - stylized document used in plotting to control visualization theme.

0.3 Code Management via Github

The project leader was responsible for reviewing all pull requests, ensuring that the code followed the project guidelines, and merging the code into the main branch. The team coordinated through a WhatsApp group chat and would regularly check in on each other's progress. Everyone was informed when code was merged so we could all pull the latest changes.

Most times, approving and merging was straightforward with no issues, but there was an interesting challenge when multiple uploads from different team members overlapped, causing conflicts. This forced us to learn about stashing and rebasing to the newly updated master branch before recommitting. The following process was ironed out to ensure a smooth integration:

1. Stash local (uncommitted) changes
`|| git stash`
2. Pull latest changes from master branch
`|| git checkout master git pull main master`
3. Rebase local changes onto master branch so it starts after the new merged code
`|| git checkout <fork-feature>/<fork-branch> git rebase master`
4. Resolve any conflicts & re-apply stashed changes
`|| git stash pop`
5. Commit and push (using `--force` parameter due to rebase)
`|| git push <fork-remote> <fork-branch> --force`

0.4 Makefile

As the project evolved, so did the Makefile. Here are some of the features we implemented:

Starting off, there are few main variables that control the compilation process....
`|| FC = gfortran`
`FFLAGS = -O2 -Wall -Wextra OBJ_DIR = ../bin EXE = (OBJ_DIR)/main_serial.x LIB_OBJ=(OBJ_DIR)/`
`\ (OBJ_DIR)/io.o\...MAIN_OBJ=(OBJ_DIR)/main_serial.o`
`NP ?= 4 OMP_THREADS ?= 2`

- **FC:** Stands for “Fortran Compiler”. We tell it to use gfortran.
- **FFLAGS:** These are the compiler flags. `-O2` is the optimization schema, and `-Wall -Wextra` enables warnings.
- **OBJ_DIR & EXE:** Defines the compiled binaries directory and the name of the final executable.
- **LIB_OBJ & MAIN_OBJ:** Defines the object files for the library and the main program.
- **NP & OMP_THREADS:** Defines the number of processes (MPI) and threads (openMP) to use for the parallel version of the program. The default values are set to 4 and 2, respectively, but they can be overridden by setting them when compiling in the command line (e.g., `make run_parallel NP=7 OMP_THREADS=3`).

In order to encompass both the serial and parallel versions of the program, a check is performed on the `make` command entered on the command-line for the word `parallel`. If found, the user's system is searched to ensure the environment has the necessary MPI and OpenMP libraries installed. If not, an error message is printed and the program exits. If they are installed, the

compiler variable `FC` is overwritten and the MPI and OpenMP flags are added to the `FFLAGS` variable.

```

[] ifneq (, (findstringparallel, (MAKECMDGOALS))) # MPI Compiler Wrapper Verification
MPI_BIN := (shellcommand-vmCIF902 > /dev/null) ifeq ((strip (MPI_BIN)), ) (error "MPI
Compiler is required but 'mpif90' was not found! Please install it (e.g. 'sudo apt install openmpi-
bin')") endif # Overwrite Compiler for MPI parallelization flag linking FC = mpif90 ...

```

Instead of writing a rule for every single `.f90` file, a pattern rule (with `%`) is used.

```

[] (OBJ_DIR)/%.o : lib/%.f90@mkdir-p(OBJ_DIR) (FC)(FFLAGS) -J(OBJ_DIR) -I(OBJ_DIR)
-c < -o@

```

- It says: “To make any `.o` file in the object directory (`bin/`) from a matching `.f90` file in `lib/`, run this command.”
- `-c $<`: Compiles the “first dependency” (`$<`, which is the `.f90` file) without linking (because that’s done in the `$(EXE)` target).
- `-J$(OBJ_DIR) -I$(OBJ_DIR)`: Tells Fortran to put module `.mod` files in the `bin/` folder.

The following is an example of the `all` target (serial version of the program) with its executable structure, which we then follow up with the explicit dependencies to create the `.o` files for the library modules and the main program

```

[] all: (EXE)

```

```

(EXE) : (LIB_OBJ) (MAIN_OBJ)@mkdir-p(OBJ_DIR) (FC)(FFLAGS) -o @(LIB_OBJ)
(MAIN_OBJ)

```

```

(OBJ_DIR)/energy.o : lib/energy.f90\ (OBJ_DIR)/parameters.o ...

```

```

(OBJ_DIR)/main_serial.o : main_serial.f90(LIB_OBJ) @mkdir -p (OBJ_DIR)(FC)
(FFLAGS) -J(OBJ_DIR) -I(OBJ_DIR) -c < -o @ ...

```

- `all`: This is the default target that needs `$(EXE)` to exist.
- `$(EXE)`: To build the final executable, it needs all the `.o` files from the `../bin` folder, which it makes sure exists (`mkdir -p`), and then links them all together using `gfortran` or `mpif90` to output (`-o $@`) the final program.

– **Note:** The `@` symbol is used 2 different ways here:

1. Before the `mkdir` command to suppress being printed to the terminal.
2. As an automatic variable representing the target name (`$(EXE)` in this case).

Last, we declare the shortcut commands as `.PHONY` targets to prevent conflicts with files that may have the same name as a target.

```

[] .PHONY: all clean run figures pipeline

```

```

run: all @mkdir -p ../results (EXE)

```

```

figures: python lib/plot_results.py

```

```

clean: rm -rf (OBJ_DIR)/ *.o(OBJ_DIR)/ *.mod (EXE)

```

```

pipeline: (MAKE)run(MAKE) figures

```

- This enables the following compile-time shortcuts:

– **make run**: Automatically builds the code (`all`) and then executes it.

- **make figures:** Envokes python to generate the plots from the `../results` folder.
- **make clean:** Acts like a reset button, deleting all compiled files to start fresh.
- **make pipeline:** A neat wrapper we built to run the code and generate the python plots back-to-back.

0.5 Shell Scripting

Itxaso to add her bit here

A qsub script `run_collab.qsub` was also written during the testing of the program to submit an individual portion of the simulation to the HPC cluster. It involved loading the appropriate modules for that queue, selecting a custom makefile (separate from the main program’s Makefile), indentifying the correct commandline arguments to pass at compile time, and submit the mpirun command.

1 Initial Configuration Generation

1.1 Internal-Coordinate Builder

The `initial_conf` module is responsible for generating the starting conformation of the polyethylene chain. The builder uses a recursive internal-coordinate approach: each successive carbon atom is placed by specifying a fixed C–C bond length ($d = 1.54 \text{ \AA}$), a fixed C–C–C bond angle ($\theta = 114^\circ$), and a selectable dihedral angle ϕ . A hard-sphere overlap check with threshold $r_{\min} = 2.0 \text{ \AA}$ is performed after each placement to avoid unphysical self-intersections during chain construction.

```
! Place carbon i based on the last two accepted positions
do i = 4, n_carbons
  ! Bond vector from i-2 to i-1 (normalised)
  b1 = coords(i-1,:) - coords(i-2,:)
  b1 = b1 / vnorm(b1)

  ! Normal to the C(i-2)-C(i-1)-C(i) plane
  b0 = coords(i-2,:) - coords(i-3,:)
  n_vec = cross(b0, b1)
  if (vnorm(n_vec) > 1.0d-8) n_vec = n_vec / vnorm(n_vec)

  ! Proposed new position using dihedral phi
  d_vec = bond_length * (
    -cos_theta * b1
    + sin_theta * (cos_phi * n_vec
                  + sin_phi * cross(b1, n_vec)))

  candidate = coords(i-1,:) + d_vec

  ! Overlap check against all previously placed atoms
  overlap = .false.
  do j = 1, i-1
    if (vnorm(candidate - coords(j,:)) < r_min) then
      overlap = .true.
      exit
    end if
  end do
  if (.not. overlap) coords(i,:) = candidate
end do
```

Listing 1: Core internal-coordinate placement loop in `initial_conf.f90`.

When explicit hydrogens are requested (`explicit_h = .true.`), two hydrogen atoms are appended for each internal CH_2 group and one for each terminal CH_3 group, giving $N_{\text{atoms}} = 3N_C + 2$ total atoms for a chain of N_C carbons. The hydrogen placement is based on a local tetrahedral frame: instead of placing the H atoms coplanar with the C–C–C backbone (chemically inconsistent), they are positioned out of the backbone plane at the correct tetrahedral angle of $\approx 109.5^\circ$.

```
! For each internal carbon i (not first or last):
! Build local orthonormal frame {b_fwd, perp, out_of_plane}
b_fwd = coords(i+1,:) - coords(i-1,:)
b_fwd = b_fwd / vnorm(b_fwd)

! Perpendicular to backbone, in the C(i-1)-C(i)-C(i+1) plane
b_back = coords(i-1,:) - coords(i,:)
b_back = b_back / vnorm(b_back)
perp = b_back - sum(b_back*b_fwd)*b_fwd
if (vnorm(perp) > 1.0d-8) perp = perp / vnorm(perp)

! Out-of-plane direction (cross product)
out_of_plane = cross(b_fwd, perp)
out_of_plane = out_of_plane / vnorm(out_of_plane)

! Place H1 and H2 symmetrically out of plane (tetrahedral geometry)
! cos(109.5 deg) ~ -1/3 => sin(109.5 deg) ~ sqrt(8/9)
h1 = coords(i,:) + r_CH * ( (-1.0d0/3.0d0)*(-perp) &
+ sqrt(8.0d0/9.0d0) * 0.5d0 * ( out_of_plane + b_fwd) )
h2 = coords(i,:) + r_CH * ( (-1.0d0/3.0d0)*(-perp) &
+ sqrt(8.0d0/9.0d0) * 0.5d0 * (-out_of_plane + b_fwd) )
```

Listing 2: Out-of-plane hydrogen placement for internal CH_2 groups.

1.2 Configuration Modes

Five distinct initial configuration modes have been implemented, each designed to probe a different region of the polymer’s conformational space:

Table 1: Summary of implemented initial configuration types.

conf_type	Name	Description
1	All-trans (planar)	All dihedrals set to $\phi = 0^\circ$; fully extended zigzag chain. Maximum end-to-end distance.
2	Random	Each dihedral drawn uniformly from $[-\pi, \pi]$; maximally disordered starting state.
3	RIS-like discrete	Dihedrals assigned with probabilities reflecting the Rotational Isomeric State model (trans, gauche ⁺ , gauche ⁻).
4	Spring / helix	Progressive dihedral increment per bond, producing a helical backbone (proposed by ai-murphy).
5	Perturbed trans	Small random perturbation applied to the all-trans geometry; slight out-of-plane deviation while remaining close to the global minimum.

Configurations 1 and 5 span the near-trans region of phase space, whereas configurations 2 and 3 provide disordered starting points. Configuration 4 was added to test helical initial geometries. This variety is relevant when the simulation is tested at different temperatures: some initial states may lie closer to the equilibrated ensemble than others, reducing the effective burn-in time.

2 Input/Output Module

2.1 Parameter Parsing from `input.dat`

All simulation parameters are read from a plain-text file `input.dat` through the `io` module. The parser applies default values for any missing keyword, strips inline comments (lines beginning with `#`), and performs basic range validation. The following parameters are controlled from the input file:

Table 2: Simulation parameters read from `input.dat`.

Parameter	Default	Description
<code>n_carbons</code>	10	Number of backbone carbon atoms
<code>n_steps</code>	1,000,000	Number of MC production steps
<code>temperature</code>	300.0	Simulation temperature (K)
<code>conf_type</code>	1	Initial configuration mode (see Table ??)
<code>explicit_h</code>	<code>.false.</code>	Enable all-atom (explicit hydrogen) mode
<code>rng_seed</code>	42	Random number generator seed
<code>print_interval</code>	1,000	Steps between output writes
<code>max_delta</code>	0.5	Maximum dihedral rotation per MC step (rad)

Output is written in XYZ format: trajectories to `traj_label.xyz`, energies to `energy_label.dat`, and observables to `observables_label.dat`, where the label encodes the key simulation parameters (`n_carbons`, `conf_type`, `n_steps`, `temperature`) for unambiguous identification.

3 Energy Evaluation and Monte Carlo Engine

3.1 Energy Calculations